Last revision: 20260901

Divide and Conquer - Sorting

CSC 3449

Rodrigo Canaan
Adapted with permission from Theresa Migler

Divide and Conquer

Divide and Conquer is a strategy that solves a problem by:

1) Breaking the problem into subproblems that are themselves smaller instances of the same type of problem.
2) Recursively solving these subproblems.
3) Appropriately combining their answers.

Sorting

Sorting

Input: A list of numbers, $a_1, a_2, a_3, \ldots a_n$.
Goal: Return a list of the same numbers sorted in increasing order.

Example:

Given: 4, 907, 34, 18, 42, 36, 71, 34, 16

Return: 4, 16, 18, 34, 34, 36, 42, 71, 907

SelectionSort

Sorting

Input: A list of numbers, $a_1, a_2, a_3, \ldots a_n$.
Goal: Return a list of the same numbers sorted in increasing order.

We will start with a straightforward algorithm:

SelectionSort($A[1, \ldots, n]$)

Input: A list of unsorted numbers $A[1, \ldots, n]$

Output: The same list sorted in increasing order

1: **for** $i = 1, \ldots, n$ **do**
2: Find min of $A[i, \ldots, n-1]$.
3: Suppose that the min occurs at position j.
4: Swap $A[i]$ with $A[j]$.

SelectionSort - Correctness

Lemma

Upon completion of SelectionSort*, for any $i \in \{1, \ldots, n-1\}$, $A[i] \leq A[i+1]$. Furthermore, all elements of the input list are in A.*

Proof.

Suppose, for a contradiction, that there is a j such that upon completion of SelectionSort, $A[j-1] > A[j]$.
Let $A[j-1] = x$ and $A[j] = y$.
At iteration $j-1$ of the algorithm, $A[j-1]$ was set to be x (step 4).
Thus, at iteration $j-1$, x must have been the smallest remaining element (step 2).
Contradiction. Because y was a remaining element at iteration $j-1$ and $y < x$.

How can we show that all elements of the input list are in A?

SelectionSort - Running Time

Running Time:

- $T(n) = T(n-1) + O(n) = O(n^2)$

MergeSort

Sorting

Input: A list of numbers, $a_1, a_2, a_3, \ldots a_n$.
Goal: Return a list of the same numbers sorted in increasing order.

MergeSort($A[1, \ldots, n]$)

Input: A list of unsorted numbers $A[1, \ldots, n]$

Output: The same list, sorted in increasing order

1: **if** $n \leq 1$ **then**
    return A
2: **Else**
    3: $LeftSorted \leftarrow MergeSort\left(A\left[1, \ldots, \lfloor \frac{n}{2} \rfloor\right]\right)$
    4: $RightSorted \leftarrow MergeSort(A\left[\lfloor \frac{n}{2} \rfloor + 1, n\right])$
    5: return $Merge(LeftSorted, RightSorted)$

MergeSort

Sorting

Input: A list of numbers, $a_1, a_2, a_3, \ldots a_n$.
Goal: Return a list of the same numbers sorted in increasing order.

Or, more concisely,

MergeSort($A[1, \ldots, n]$)

Input: A list of unsorted numbers $A[1, \ldots, n]$

Output: The same list, sorted in increasing order

1: **if** $n \leq 1$ **then**
    return A
2: **else**
    return *Merge*(*MergeSort*($A[1, \ldots, \lfloor n/2 \rfloor]$), *MergeSort*($A[\lfloor n/2 \rfloor+1, \ldots, n]$))

MergeSort

Merge (subproblem of sorting)

Input: Two sorted sequences of numbers, $x = x_1, x_2, \ldots, x_\ell$ and $y = y_1, y_2, \ldots, y_m$.
Goal: Return a sorted list containing all elements of x, y

Merge(x[1, ..., ℓ], y[1, ..., m])

Input: Two sorted lists, x[1, ..., ℓ] and y[1, ..., m]

Output: One sorted list that contains all elements of both lists.

1: **if** $x = \emptyset$ **then return** y

2: **if** $y = \emptyset$ **then return** x

3: **if** $x[1] \leq y[1]$ **then**

    return $x[1] \circ$ *Merge*(x[2, ..., ℓ], y[1, ..., m])

4: **else**

    return $y[1] \circ$ *Merge*(x[1, ..., ℓ], y[2, ..., m])

MergeSort - Correctness

As usual, we must ask ourelves two questions:

- Is it correct?
- What is its running time?

MergeSort - Correctness

Let's start by showing Merge is correct

Lemma 1

Given two sorted lists, x and y, of total size n Merge returns a sorted list containing all elements from *x and y, for all n*

Proof of Lemma 1.

We will procede by induction on the total size of the lists being merged.

- **Base Case:** ($n = 1$)
- This will only occur if either *x* or *y* is empty, and the other list has exactly 1 element.
  - Merge correctly merges the empty list with any other sorted list (steps 1 and 2).

MergeSort - Correctness

Proof of Lemma 1 (Cont.)

- **Inductive Hypothesis:** Suppose that, given two sorted lists, *x* and *y*, of total size *k,* Merge returns a sorted list containing all elements from *x* and *y*.

- **Inductive Step:** Consider two sorted lists with total size *k*+1. We must show that Merge correctly
  - In steps 3 and 4 of the algorithm, Merge correctly places the smallest element at the beginning of the list.
  - Merge then concatenates that element with the Merge of the remaining elements of the two lists.
    - The total size of the remaining two lists is *k*.
    - By the Inductive Hypothesis, Merge correctly merges the remainder.
  - This concludes the inductive step: Merge correctly merges two sorted lists of total size *k+1*

- **Conclusion:** Therefore, by PMI, Merge correctly merges two sorted lists.

MergeSort - Correctness

Theorem

Given a list A of size n, MergeSort returns a sorted list with all elements of A

Base Case: (*n* = 0 or n 1)

An empty list or a singleton is already sorted. MergeSort correctly returns the original list (step 1)

Inductive Hypothesis

Suppose MergeSort correctly sorts all lists of size $n \leq k$

Inductive Step

Consider a list A with $k + 1$ elements.

In steps 3 and 4, the algorithm recursively sorts the first and second halves of A.

By inductive Hypothesis, both halves are correctly sorted since they each have size $\leq k$

Since *Merge* correctly merges any two sorted lists (Lemma 1), the returning list is also sorted, and together they contain all elements of A

This concludes our inductive step, and MergeSort correctly sorts lists of any size

MergeSort - Running Time

What is the running time?

- $T(n) = 2T(n/2) + O(n) = O(n \log n)$